Operating System (ISE3137)

Jump Together, Fly Farther!

School of Global Convergence Studies

인하대학교 국제학부

Week 11 Lab

ISE Department

Prof. Mehdi Pirahandeh

Storage management concepts

A file system is an organized structure of data-holding files and directories residing on a storage device, such as a physical disk or partition. The file system hierarchy discussed earlier assembles all the file systems into one tree of directories with a single root, the / directory. The advantage here is that the existing hierarchy can be extended at any time by adding a new disk or partition containing a supported file system to add disk space anywhere in the file system tree. The process of adding a new file system to the existing directory tree is called *mounting*. The directory where the new file system is mounted is referred to as a *mount point*. This is a fundamentally different concept than that used on a Microsoft Windows system, where a new file system is represented by a separate drive letter.

Hard disks and storage devices are normally divided up into smaller chunks called *partitions*. A partition is a way to compartmentalize a disk. Different parts of it can be formatted with different file systems or used for different purposes. For example, one partition could contain user home directories while another could contain system data and logs. If a user fills up the home directory partition with data, the system partition may still have space available. Placing data in two separate file systems on two separate partitions helps in planning data storage.

ACCESSING LINUX FILE SYSTEMS

인하대학교 INHA UNIVERSITY

Hard disks and storage devices are normally divided up into smaller chunks called *partitions*. A partition is a way to compartmentalize a disk. Different parts of it can be formatted with different file systems or used for different purposes. For example, one partition could contain user home directories while another could contain system data and logs. If a user fills up the home directory partition with data, the system partition may still have space available. Placing data in two separate file systems on two separate partitions helps in planning data storage.

Storage devices are represented by a special file type called *block device*. The block device is stored in the **/dev** directory. In Red Hat Enterprise Linux, the first SCSI, PATA/SATA, or USB hard drive detected is **/dev/sda**, the second is **/dev/sdb**, and so on. This name represents the whole drive. The first primary partition on **/dev/sda** is **/dev/sda1**, the second partition is **/dev/sda2**, and so on.

A long listing of the **/dev/vda** device file on serverX reveals its special file type as **b**, which stands for block device:

```
[student@serverX ~]$ ls -l /dev/vda
brw-rw----. 1 root disk 253, 0 Mar 13 08:00 /dev/vda
```

Note

Exceptions are hard drives in virtual machines, which typically show up as **/dev/vd<*letter*>** or **/dev/xvd<*letter*>**.

ACCESSING LINUX FILE SYSTEMS

인하대학교 INHA UNIVERSITY

Another way of organizing disks and partitions is with *logical volume management* (LVM). With LVM, one or more block devices can be aggregated into a storage pool called a *volume group*. Disk space is made available with one or more *logical volumes*. A logical volume is the equivalent of a partition residing on a physical disk. Both the volume group and the logical volume have names assigned upon creation. For the volume group, a directory with the same name as the volume group exists in the **`/dev`** directory. Below that directory, a symbolic link with the same name as the logical volume has been created. For example, the device file representing the **`mylv`** logical volume in the **`myvg`** volume group is **`/dev/myvg/mylv`**.

It should be noted that LVM relies on the *Device Mapper* (DM) kernel driver. The above symbolic link **`/dev/myvg/mylv`** points to the **`/dev/dm-`*`number`*** block device node. The assignment of the *number* is sequential beginning with zero (0). There is a another symbolic link for every logical volume in the **`/dev/mapper`** directory with the name **`/dev/mapper/myvg-mylv`**. Access to the logical volume can generally use either of the consistent and reliable symbolic link names, since the **`/dev/dm-`*`number`*** name can vary with each boot.

Examining file systems

To get an overview about the file system mount points and the amount of free space available, run the **df** command. When the **df** command is run without arguments, it will report total disk space, used disk space, and free disk space on all mounted regular file systems. It will report on both local and remote systems and the percentage of the total disk space that is being used.

Display the file systems and mount points on the serverX machine.

```
[student@serverX ~]$ df
Filesystem     1K-blocks    Used Available Use% Mounted on
/dev/vda1        6240256 4003760   2236496  65% /
devtmpfs          950536       0    950536   0% /dev
tmpfs             959268      80    959188   1% /dev/shm
tmpfs             959268    2156    957112   1% /run
tmpfs             959268       0    959268   0% /sys/fs/cgroup
```

The partitioning on the serverX machine shows one real file system, which is mounted on **/**. This is common for virtual machines. The *tmpfs* and *devtmpfs* devices are file systems in system memory. All files written into tmpfs or devtmpfs disappear after system reboot.

ACCESSING LINUX FILE SYSTEMS

인하대학교 INHA UNIVERSITY

To improve readability of the output sizes, there are two different *human-readable* options: **-h** or **-H**. The difference between these two options is that **-h** will report in KiB (2^{10}), MiB (2^{20}), or GiB (2^{30}), while the **-H** option will report in SI units: KB (10^{3}), MB (10^{6}), GB (10^{9}), etc. Hard drive manufacturers usually use SI units when advertising their products.

Show a report on the file systems on the serverX machine with all units converted to human-readable format:

```
[student@serverX ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1       6.0G  3.9G  2.2G  65% /
devtmpfs        929M     0  929M   0% /dev
tmpfs           937M   80K  937M   1% /dev/shm
tmpfs           937M  2.2M  935M   1% /run
tmpfs           937M     0  937M   0% /sys/fs/cgroup
```

ACCESSING LINUX FILE SYSTEMS

인하대학교 INHA UNIVERSITY

For more detailed information about the space used by a certain directory tree, there is the **du** command. The **du** command has **-h** and **-H** options to convert the output to human-readable format. The **du** command shows the size of all files in the current directory tree recursively.

Show a disk usage report for the **/root** directory on serverX:

```
[root@serverX ~]# du /root
4 /root/.ssh
4 /root/.cache/dconf
4 /root/.cache
4 /root/.dbus/session-bus
4 /root/.dbus
0 /root/.config/ibus/bus
0 /root/.config/ibus
0 /root/.config
14024 /root
```

Show a disk usage report in human-readable format for the **/var/log** directory on serverX:

```
[root@serverX ~]# du -h /var/log
...
4.9M /var/log/sa
68K /var/log/prelink
0 /var/log/qemu-ga
14M /var/log
```

ACCESSING LINUX FILE SYSTEMS

인하대학교 INHA UNIVERSITY

Description	Device file
The device file of a SATA hard drive residing in **/dev**.	/dev/sdc
The device file of the second partition on the first SATA hard drive in **/dev**.	/dev/sda2
The device file of a logical volume in **/dev**.	/dev/vg_install/lv_home
The device file of the second disk in a virtual machine in **/dev**.	/dev/vdb
The device file of the third partition on the second SATA hard drive in **/dev**.	/dev/sdb3
The device file of the third partition on the second disk in a virtual machine in **/dev**.	/dev/vdb3

Mounting and Unmounting File Systems

Mounting file systems manually

A file system residing on a SATA/PATA or SCSI device needs to be mounted manually to access it. The **mount** command allows the root user to manually mount a file system. The first argument of the **mount** command specifies the file system to mount. The second argument specifies the target directory where the file system is made available after mounting it. The target directory is referred to as a mount point.

The **mount** command expects the file system argument in one of two different ways:

- The device file of the partition holding the file system, residing in **/dev**.
- The *UUID*, a universal unique identifier of the file system.

Note

As long as a file system is not recreated, the UUID stays the same. The device file can change; for example, if the order of the devices is changed or if additional devices are added to the system.

Mounting and Unmounting File Systems

The **blkid** command gives an overview of existing partitions with a file system on them and the UUID of the file system, as well as the file system used to format the partition.

```
[root@serverX ~]# blkid
/dev/vda1: UUID="46f543fd-78c9-4526-a857-244811be2d88" TYPE="xfs"
```

Note

A file system can be mounted on an existing directory. The **/mnt** directory exists by default and provides an entry point for mount points. It is used for manually mounting disks. It is recommended to create a subdirectory under **/mnt** and use that subdirectory as a mount point unless there is a reason to mount the file system in another specific location in the file system hierarchy.

Mount by device file of the partition that holds the file system.

```
[root@serverX ~]# mount /dev/vdb1 /mnt/mydata
```

Mount the file system by universal unique id, or the UUID, of the file system.

```
[root@serverX ~]# mount UUID="46f543fd-78c9-4526-a857-244811be2d88" /mnt/mydata
```

Mounting and Unmounting File Systems

Note

If the directory acting as mount point is not empty, the files that exists in that directory are not accessible as long as a file system is mounted there. All files written to the mount point directory end up on the file system mounted there.

Unmounting file systems

To unmount a file system, the **umount** command expects the mount point as an argument.

Change to the **/mnt/mydata** directory. Try to umount the device mounted on the **/mnt/mydata** mount point. It will fail.

```
[root@serverX ~]# cd /mnt/mydata
[root@serverX mydata]# umount /mnt/mydata
umount: /mnt/mydata: target is busy.
        (In some cases useful info about processes that use
         the device is found by lsof(8) or fuser(1))
```

Mounting and Unmounting File Systems

The **lsof** command lists all open files and the process accessing them in the provided directory. It is useful to identify which processes currently prevent the file system from successful unmounting.

```
[root@serverX mydata]# lsof /mnt/mydata
COMMAND  PID USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
bash    1593 root  cwd    DIR  253,2        6  128 /mnt/mydata
lsof    2532 root  cwd    DIR  253,2       19  128 /mnt/mydata
lsof    2533 root  cwd    DIR  253,2       19  128 /mnt/mydata
```

Once the processes are identified, an action can be taken, such as waiting for the process to complete or sending a SIGTERM or SIGKILL signal to the process. In this case, it is sufficient to change the current working directory to a directory outside the mount point.

```
[root@serverX mydata]# cd
[root@serverX ~]# umount /mnt/mydata
```

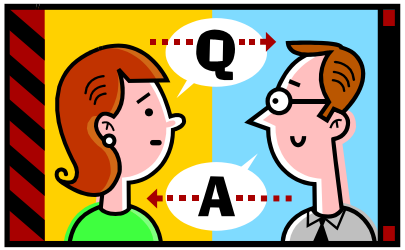
Note

A common cause for the file system on the mount point to be busy is if the current working directory of a shell prompt is below the active mount point. The process accessing the mount point is **bash**. Changing to a directory outside the mount point allows the device to be unmounted.

Brief break (if on schedule)

인하대학교 INHA UNIVERSITY

Q&A?



Prof. Mehdi Pirahandeh
E-mail : mehdi@inha.ac.kr